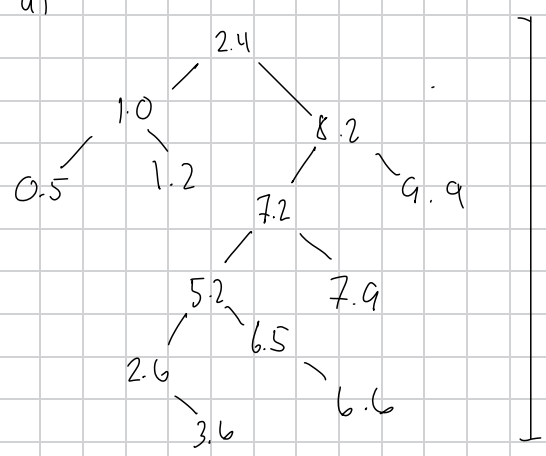
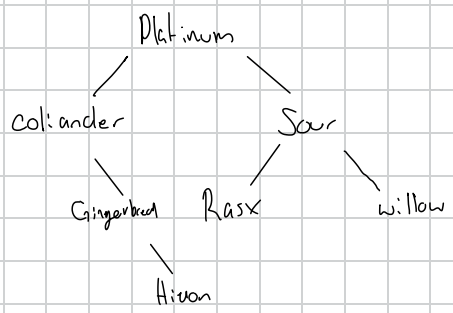
1) text (h = height of tree)

a)



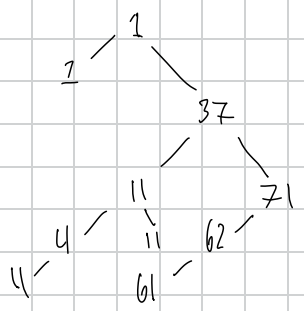
h = 5

b)



h = 3

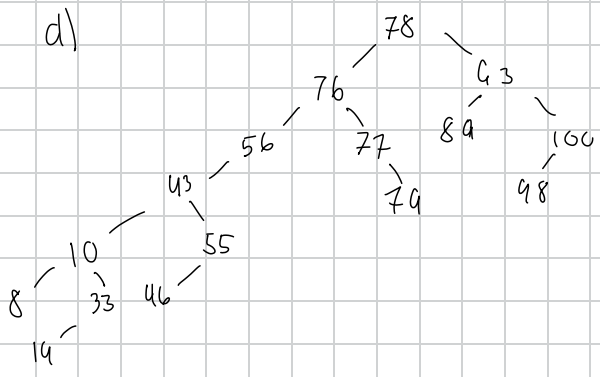
c)



h = 4

- deall with duplicates by checking if either child is null, then if true place it there. If not follow: if (node ≤ root):
  recurse (root.left)
  ↳ would entail saving keys in set to check for duplicates

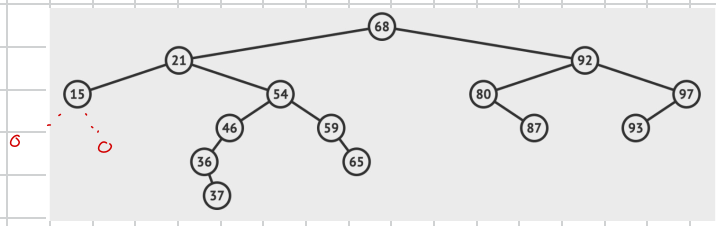
d)



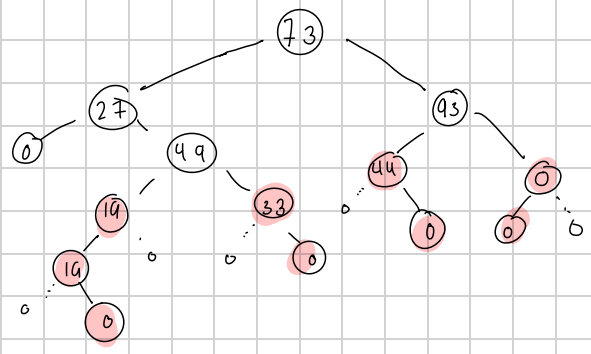
h = 6

---

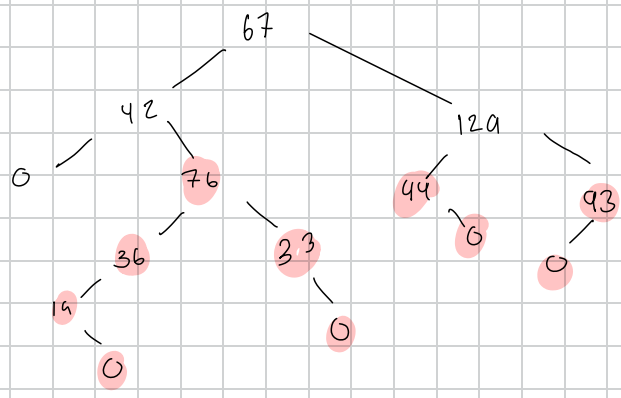
2) text



T' (Inorder)



T'' (preorder)



b) T' & T'' are not BSTs due to breaking the BST:

left < node < right.

The rule breaking nodes can be seen highlighted red.

c) Since both T' & T'' are not BST they are also not AVL tree since AVL trees are self balancing BSTs that follow all BST rules.

5) All less

time complexity:

The time complexity would be $O(n)$ where n is the number of strings in the given maxheap because the program does a constant number of operations n times. For the program Big-O is equal to Big-Ω, because the program will always do n amount of constant operations.

The space complexity would be $O(n)$ where n is the number of strings in the max heap, because we create "results" ArrayList, which can grow up to n size. Big-O = Big Ω for space as well.

4) Encrypted Binary Tree

The time complexity would be $O(n)$ where n is the number of nodes in the given tree, because worst case the target number is not found in the given tree and the program does the recursive calls and all nodes. For the best case the time complexity would be $\Omega(1)$, because if any of the base cases are true, no other calls are made.

The space complexity is $\Theta(1)$ since the auxiliary space does not change depending on the input.

6) deleteInRange();

The time complexity would be $O(n \log n)$ where n is the number of nodes in the tree because deletion is $O(\log n)$ (cost of traversing the tree), and in the worst case all nodes are in the range of deletion therefore we would do the remove() operation a n amount of times, hence $O(n \log n)$. In the best case none of the nodes would be in range and the time complexity would be $\Omega(\log n)$ since we would simply be transversing a balanced tree.

The space complexity would be $O(1)$ because the number of created variables for the operation is constant.

3) Huffman code:

Pf: In the frequency Count() method the for loop runs for s times, where s is the length of the given string. Therefore the time complexity is $O(s)$.

The method buildHuffman() contains a for loop runs for n times where n is the amount of unique elements in a given string, because the frequencyMap allows for a unique set of nodes, meaning no duplicates are allowed.

The while loop that exist in the function runs for $n-1$ times, and has the poll() operation that has a time complexity of $O(\log n)$ because removing the root mean the heap must heapify down the tree which has a time complexity of $\log n$. Therefore you must multiply $n \times \log n$, hence the buildHuffman has a time complexity of $(n + n \log n)$.

The genCode function has a time complexity of $O(m)$, where m is the number of nodes in the huffman tree, because the recursive function is called on all non-leaf nodes in the tree and if the node is a leaf then it is put into the hashmap codes.

encode() has a time complexity of $O(s)$ because it contains a for loop that runs for s times

The function print stats has a time complexity of $O(2n+s)$ because it contains 2 for loops that run n times and one for loop that runs s times.

Pf: (space)

The space complexity of frequencyCount would be O(n) because the function creates a constant number of variables but adds a n amount of entries to the frequency map.

The space complexity of buildHuffman O(n) because the function creates n number of tree Nodes. Now consider the while loop, this does not have any effect because, even though, there are parent node being created, $\frac{n}{2}$ nodes, the constant rule can be used and we get n+n, which could just be denoted O(n).

The space complexity of genCode() would be O(1) because there is a constant number of variables being created, and the input does not affect this.

The space complexity of encode would be O(s) because result string length depends on the number of characters in the string.

The space complexity of print stats would be O(1) because the number of variables stays constant.